Synapse.

Time is brain

91Life | Feel Change.

Albanian Skills
2026

Software Development
Project Statement

Synapse: Building a Stroke Command Center

Overview

This year you build **Synapse**, the engine of a stroke command center for Albania, as a command-line program.

A stroke happens when a clot blocks the blood going to part of the brain. That part stops working within minutes, and every minute without treatment costs about 1.9 million brain cells. Doctors call this "time is brain".

Synapse follows one patient from the first symptom to recovery. It works out what is wrong, where to drive the patient, when they get treated and how they recover.

You do not need any medical knowledge. Every rule you need is written in this document in plain words.

You receive the missions one part at a time during the competition. Each part builds on the code you have already written.

You build one program, mission by mission. Each mission adds new commands to the same program and saves its results for the missions after it. By the end you have a working prototype of the whole engine.

What we're looking for

- **Functional, tested software.** A working system is paramount. A simple program that works beats a complex one with issues. The results matter.
- **Object-oriented design.** Use OOP to reflect the real world in your code. Pipes, brain areas, patients and hospitals should be objects.
- **Code readability and reuse.** Your code will be read by others, and by you months later. Keep it clear and simple.
- **Efficient solutions.** Pick sensible algorithms and data structures, so commands answer fast.
- **Adaptability.** During the competition we will hand out changes: a new version of a data file, or a new rule. A program that keeps its data in files and its rules in one place handles a change by loading a new file. A program with hardcoded values has to be rewritten.

How it works

Synapse is a terminal program only: no website, no frontend, no graphics. You type a command, it prints text. Every example in this document shows exactly what the screen should look like.

- **Any programming language.** Python, Java, C#, JavaScript, C++ or any other.
- **One program, many commands.** Mission 1 adds the `atlas` commands. Every later mission adds its own commands to the same program.
- **Interactive shell.** Start the program once, then type commands at a `synapse>` prompt. A program that runs one command per launch is also accepted, as long as it reloads its data every time.
- **Input files.** We give you a `data/` folder with plain text files. "Loading a file" means your program opens a file from disk by its path. Nothing is uploaded anywhere.
- **Your own database.** In a later mission, your program starts saving patients and results in a file you choose, JSON or plain text.
- **You may design the output differently** from the examples, as long as it shows the same information.

The file in `data/` for Mission 1:

File	What it holds
`atlas.txt`	How blood reaches each part of the brain

Each later mission comes with its own files. During the competition we may hand you new versions of these files. The judges also run hidden versions with tricky cases.



Mission 1: The Brain Map

The story

"Before Synapse can help a patient, it has to know how the brain gets its blood. Doctors keep this in an atlas: a map of which pipe feeds which part of the brain. Your first mission is to load that atlas into your program and let doctors ask it questions. Every later mission is built on top of it."

What you need to know

This is all the medicine Missions 1 and 2 need:

1. The heart pumps blood into pipes (blood vessels). Pipes branch into smaller pipes.
2. Each brain area does one job, for example speaking or moving the right arm. It gets blood from one or more pipes.
3. If a pipe is blocked, the pipes after it lose their blood too, unless blood can still reach them another way.
4. An area stops working only when **all** of its pipes have no blood.
5. Some pipes are big and some are small. This matters from Mission 3.

Mission 1 only stores this information. Mission 2 uses rules 3 and 4.

The base challenge

Your goal: Synapse reads the atlas from its file, catches every mistake in it, and answers a doctor's questions about where each part of the brain gets its blood.

Goal 1: Read the atlas file format

The atlas is a plain text file, `data/atlas.txt`. Its format:

- Each line is one record. Its first field says what kind: `PIPE` or `AREA`.
- Fields are separated by `|`. Spaces around a field do not count.
- Inside a field, a list of names is separated by `,`.
- Lines starting with `#` are comments. Empty lines are ignored.
- Names use lowercase letters, digits and hyphens, for example `left-middle-top`.

The two kinds of record:

```
PIPE | <name> | <size> | <comes from>
AREA | <name> | <fed by> | <symptoms if it stops working> | <what it does>
```

- `size` is `big` or `small`.
- `comes from` is one or more pipe names. Exactly one pipe, the heart, has `-` here.
- `fed by` is one or more pipe names.
- `symptoms` is one or more symptom codes, such as `cannot-speak`.

- `what it does` is free text and may contain spaces.
- A line may name a pipe that is defined further down the file.

The full file you receive:

```
# SYNAPSE BRAIN ATLAS v1
# PIPE | name | size | comes from
# AREA | name | fed by | symptoms if it stops working | what it does

PIPE | heart               | big   | -
PIPE | left-neck           | big   | heart
PIPE | left-middle         | big   | left-neck
PIPE | left-middle-top     | small | left-middle
PIPE | left-middle-bottom  | small | left-middle
PIPE | left-front          | small | left-neck
PIPE | right-neck          | big   | heart
PIPE | right-middle        | big   | right-neck
PIPE | right-middle-top    | small | right-middle
PIPE | right-middle-bottom | small | right-middle
PIPE | right-front         | small | right-neck
PIPE | back-left-neck      | small | heart
PIPE | back-right-neck     | small | heart
PIPE | back-main           | big   | back-left-neck, back-right-neck
PIPE | left-back           | small | back-main
PIPE | right-back          | small | back-main

AREA | speaking       | left-middle-top | cannot-speak | Turns thoughts into words
AREA | right-arm-face | left-middle-top | right-arm-weak, right-face-droop | Moves the right arm and face
AREA | understanding  | left-middle-bottom | cannot-understand | Understands what other people say
AREA | right-leg      | left-front | right-leg-weak | Moves the right leg
AREA | right-vision   | left-back | blind-on-right | Sees everything on the right side
AREA | left-arm-face  | right-middle-top | left-arm-weak, left-face-droop | Moves the left arm and face
AREA | left-attention | right-middle-bottom | ignores-left-side | Notices things on the left side
AREA | left-leg       | right-front | left-leg-weak | Moves the left leg
AREA | left-vision    | right-back | blind-on-left | Sees everything on the left side
AREA | memory         | left-back, right-back | forgets-new-things | Stores new memories
AREA | awake          | back-main | drowsy | Keeps the patient awake and alert
AREA | balance        | back-left-neck, back-right-neck | falls-over | Keeps balance and coordination
```

Goal 2: Model the atlas

The atlas describes real things: pipes, areas and the links between them. Design objects that mirror them. The design is yours, but the judges check that the data lives in objects loaded from the file, not in your code.

Goal 3: Load the atlas

- When the program starts, it loads `data/atlas.txt`.
- `atlas load <path>` loads another atlas file.
- **All or nothing:** if the file has any error, nothing changes and the previous atlas stays in use.
- An error message names the line where the problem is.

Example CLI interaction (you may design yours differently):

```
$ python synapse.py
Synapse 1.0 · type "help" to see all commands
SUCCESS: Atlas loaded from data/atlas.txt: 16 pipes, 12 areas.

synapse> atlas load data/atlas-broken.txt
ERROR: data/atlas-broken.txt, line 9: pipe "left-middle-top" comes from "left-midle", which does not exist.
Atlas not loaded. Still using data/atlas.txt.
```




Error handling. Your program must handle and clearly report errors such as:

- "ERROR: File not found: data/atlas2.txt"
- "ERROR: The file is empty."
- "ERROR: line 4: unknown record type "PIPES". Expected PIPE or AREA."
- "ERROR: line 6: PIPE needs 4 fields, found 3."
- "ERROR: line 7: size must be big or small, found "huge"."
- "ERROR: line 9: pipe "left-middle-top" comes from "left-midle", which does not exist."
- "ERROR: line 21: area "speaking" is fed by "left-top", which does not exist."
- "ERROR: line 22: "speaking" is defined twice (first on line 19)."
- "ERROR: The atlas needs exactly one pipe that comes from "-" (the heart). Found 0."

Goal 4: Ask the atlas questions

Add four commands:

- `atlas pipes` lists every pipe with its size and where it comes from.
- `atlas areas` lists every area with the pipes that feed it.
- `atlas show <name>` shows one pipe or one area in full. For a pipe, also print which pipes branch out of it and which areas it feeds. The file never says this directly: your program must work it out.
- `atlas find <word>` lists the areas whose name, symptoms or description contain the word, ignoring upper and lower case.

Example CLI interaction (you may design yours differently):

```
synapse> atlas pipes
NAME                 SIZE   COMES FROM
heart                big    -
left-neck            big    heart
left-middle          big    left-neck
...
back-main            big    back-left-neck, back-right-neck
16 pipes

synapse> atlas show left-middle
Pipe: left-middle
  Size:          big
  Comes from:    left-neck
  Branches into: left-middle-top, left-middle-bottom
  Feeds areas:   none directly

synapse> atlas show speaking
Area: speaking
  What it does:  Turns thoughts into words
  Fed by:        left-middle-top
  If it stops:   cannot-speak

synapse> atlas find ARM
2 areas match "arm":
  right-arm-face   Moves the right arm and face
  left-arm-face    Moves the left arm and face
```




Error handling:

- "ERROR: No pipe or area called "left-midle"."
- "ERROR: Missing name. Usage: atlas show <name>"
- "No areas match "xyz"." (a message, not a crash)

The bonus challenge

For extra points.

Bonus 1: Draw the network

`atlas tree` prints every pipe as an indented tree that starts at the heart, with each area under the pipes that feed it. A pipe or area with more than one parent is printed in full under its first parent and as `(see above)` under the others.

```
synapse> atlas tree
heart
  left-neck
    left-middle
      left-middle-top
        [speaking]
        [right-arm-face]
      left-middle-bottom
        [understanding]
    left-front
      [right-leg]
  ...
  back-left-neck
    back-main
      left-back
  ...
  back-right-neck
    back-main (see above)
```

Bonus 2: Every route to an area

`atlas path <area>` prints every route the blood can take from the heart to an area. Some areas have more than one.

```
synapse> atlas path memory
4 routes from the heart to memory:
  heart > back-left-neck > back-main > left-back > memory
  heart > back-right-neck > back-main > left-back > memory
  heart > back-left-neck > back-main > right-back > memory
  heart > back-right-neck > back-main > right-back > memory
```

Bonus 3: Deep checks

Report every error in the file at once, not only the first. Also catch:

- "ERROR: Loop: left-front comes from left-neck, which comes from left-front."
- "ERROR: Pipe "spare-pipe" can never receive blood from the heart."



What the judges will test

- The atlas you received, and a hidden `atlas-v2.txt` with more pipes and areas. Every command must work on it without changing any code.
- One broken file for each error listed above.
- Messy but valid files: extra spaces, comments, empty lines, pipes defined after the pipes that use them.

What this mission saves

Nothing goes into your database yet. The next missions work on the atlas you loaded here.